

AutoFuzzer

Embedded Protocol Robustness Testing Platform

Version 4.0

A standalone ESP32-based hardware fuzzing device that connects to external microcontrollers and tests communication handler robustness through automated, mutated packet injection across UART, SPI, and I2C protocols.

Repository:	github.com/Alwin111/AutoFuzzer
Platform:	ESP32 DevKit V1
Protocols:	UART, SPI, I2C
Targets:	STM32 Nucleo, Arduino Nano, ESP32
Tests Run:	41 campaigns across 3 boards
Test Reports:	41 reports (28 pass, 13 fail)

Table of Contents

1. Project Overview & Motivation
2. System Architecture
3. Hardware Design
4. Circuit Diagram & Pin Map
5. Firmware Architecture
6. Communication Protocol
7. Mutation Engine & Test Profiles
8. Failure Detection & Reproduction Pipeline
9. OLED User Interface
10. Web Dashboard
11. PC Companion Tool
12. Test Results & Reports
13. Unit Testing
14. Issues Faced & Solutions
15. Future Work

1. Project Overview & Motivation

What is AutoFuzzer?

AutoFuzzer is a standalone embedded protocol robustness testing platform. It uses an ESP32 microcontroller as the primary fuzzer/controller that connects to an external Device Under Test (DUT) -- such as an STM32 Nucleo, Arduino Nano, or another ESP32 -- to systematically test communication handler robustness through automated, mutated packet injection.

The system runs entirely autonomously on the ESP32. No PC is required for basic testing. An optional PC companion provides advanced features like report generation, AI-assisted debugging, and historical failure tracking.

Motivation

Embedded firmware vulnerabilities are a growing security concern. Communication parsers -- the code that processes incoming UART, SPI, and I2C data -- are among the most attack-sensitive components. A malformed packet can trigger buffer overflows, infinite loops, watchdog resets, or complete system lockups.

Manual testing of these edge cases is time-consuming and error-prone. AutoFuzzer automates this process: it generates valid, boundary, malformed, corrupted, truncated, and randomized inputs, sends them to the target, monitors the target's health, and systematically discovers robustness weaknesses.

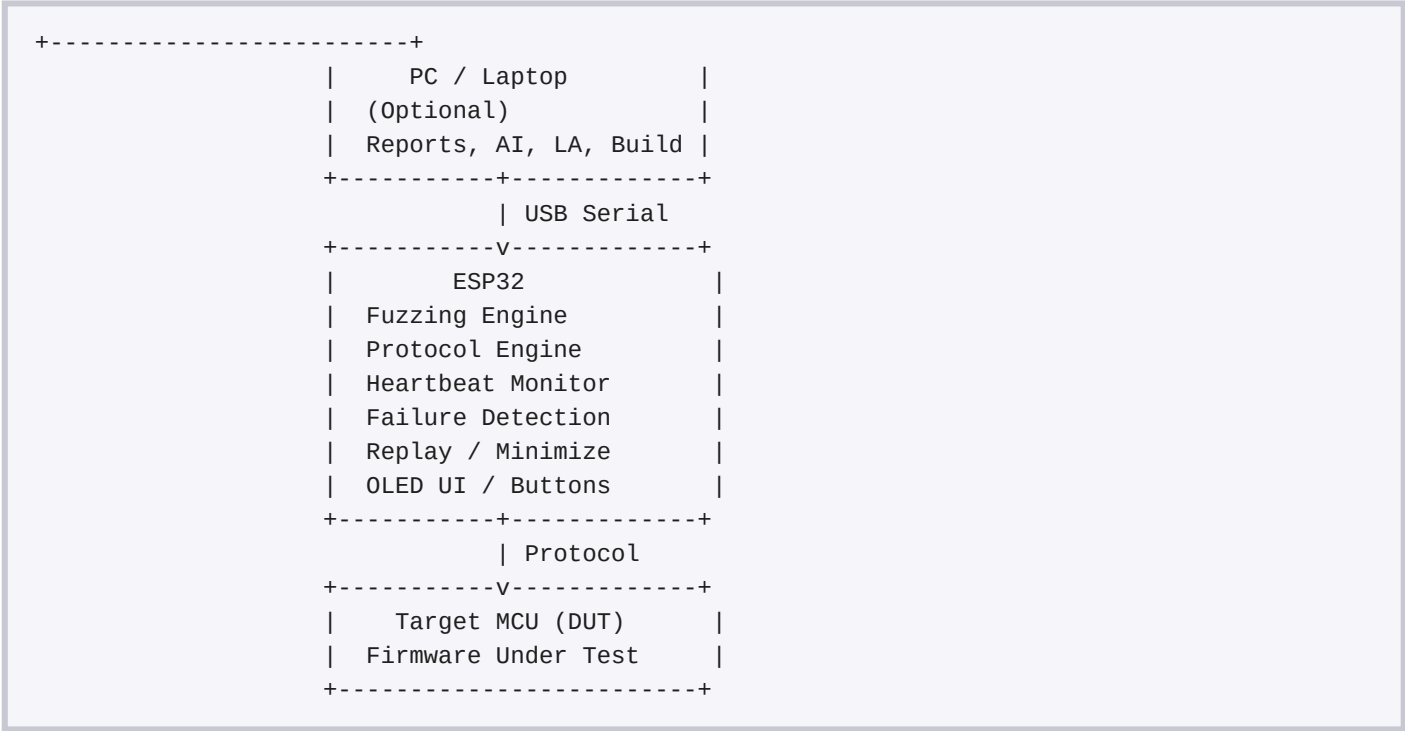
Goals

- Connect to a target MCU and test communication protocol robustness
- Generate structured test campaigns with prioritized mutation phases
- Detect failures through heartbeat monitoring and response parsing
- Freeze, replay, reproduce, and minimize failing testcases
- Generate detailed reports with robustness scores
- Provide a visual web dashboard for real-time monitoring
- Optionally analyze failures with AI and propose firmware fixes
- Create permanent regression testcases from discovered failures

2. System Architecture

Three-Layer Architecture

AutoFuzzer uses a three-layer architecture with clear separation of concerns:



Key Design Principle

STANDALONE OPERATION

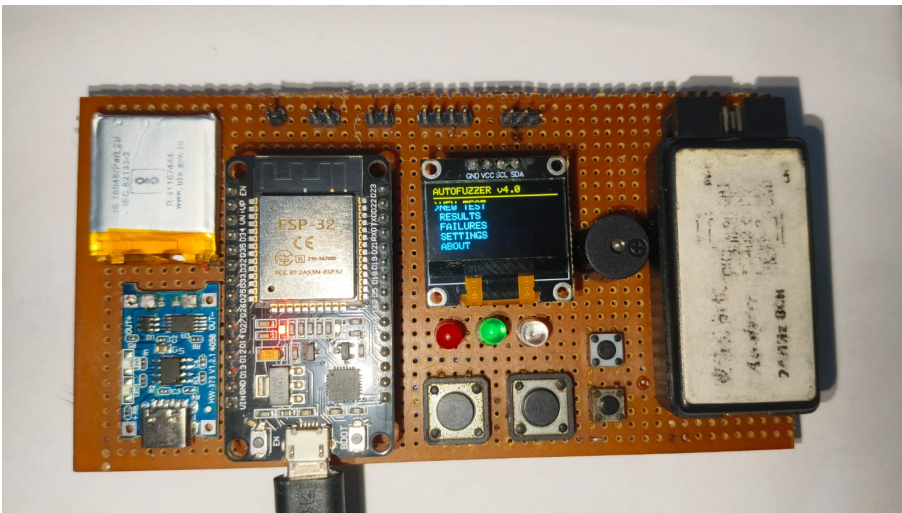
The ESP32 performs all fuzzing, monitoring, failure detection, replay, and minimization independently. The PC companion is optional -- it adds report generation, AI analysis, and historical tracking, but is never required.

3. Hardware Design

AutoFuzzer v4.0 Prototype

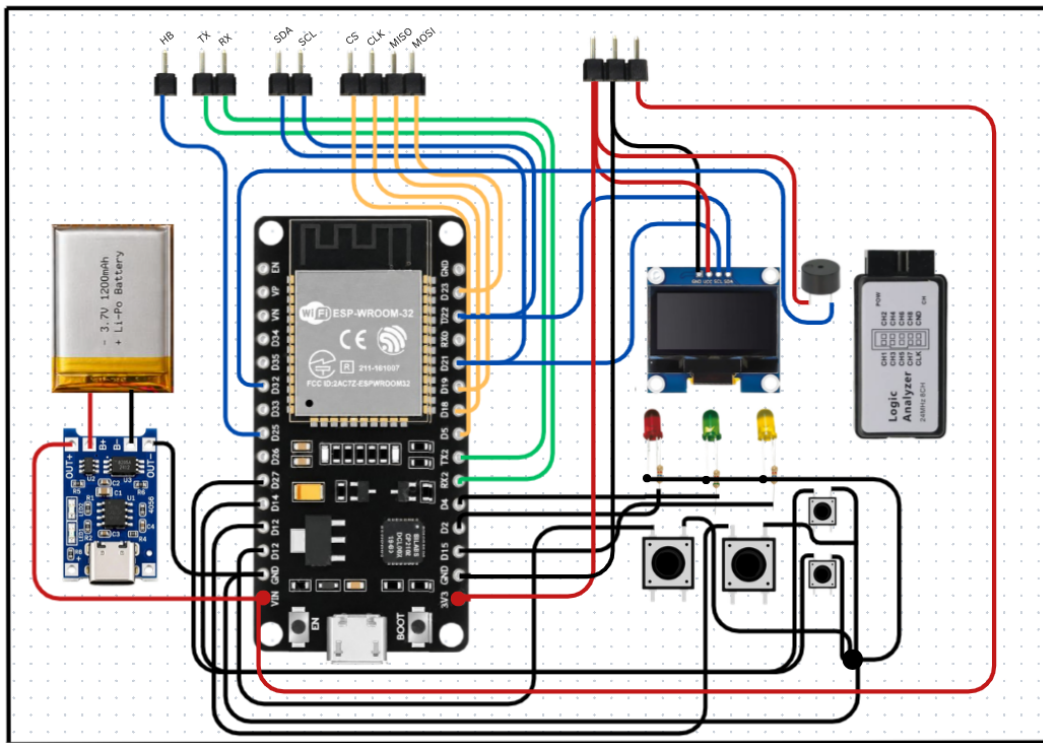
The AutoFuzzer v4.0 prototype is built on a perfboard with the following components:

- ESP32 DevKit V1 -- Main controller -- runs all fuzzing logic
- SSD1306 OLED (128x64) -- User interface -- menus, status, results
- 3x LEDs (Green/Red/Blue) -- PASS / FAIL / ACTIVE indicators
- 4x Tactile Buttons -- UP / DOWN / SELECT / BACK navigation
- Piezo Buzzer -- Audio alerts for failures
- 3.7V LiPo Battery -- Portable power source
- TP4056 Module -- Battery charging protection
- Logic Analyzer Header -- 8-channel Saleae-compatible probe connection
- Male Header Pins -- Exposed connections for DUT wiring



AutoFuzzer v4.0 prototype -- ESP32, OLED, LEDs, buttons, buzzer, battery on perfboard

4. Circuit Diagram & Pin Map



Full wiring schematic -- ESP32, OLED, LEDs, buttons, buzzer, battery, LA header

ESP32 Pin Map

GPIO	Function	Module
2	Green LED (PASS)	indicators
4	Red LED (FAIL)	indicators
5	SPI CS	spi_fuzzer
12	Button: UP	buttons
13	Button: DOWN	buttons
14	Button: BACK	buttons
15	Blue LED (ACTIVE)	indicators
16	UART RX2	uart_fuzzer / uart_parser
17	UART TX2	uart_fuzzer
18	SPI SCK	spi_fuzzer
19	SPI MISO	spi_fuzzer
21	I2C SDA / OLED	i2c_fuzzer / oled_ui
22	I2C SCL / OLED	i2c_fuzzer / oled_ui
23	SPI MOSI	spi_fuzzer

GPIO	Function	Module
25	Heartbeat Input	heartbeat
27	Button: SELECT	buttons
32	Piezo Buzzer	indicators

Wiring: ESP32 to Arduino Nano (UART)

```
ESP32 GPIO 17 (TX)  -->  Nano D3 (RX - SoftwareSerial)
ESP32 GPIO 16 (RX)  <--  Nano D2 (TX - SoftwareSerial)
ESP32 GPIO 25       <--  Nano D4 (Heartbeat)
ESP32 GND           -->  Nano GND
```

Wiring: ESP32 to STM32 Nucleo (UART)

```
ESP32 GPIO 17 (TX)  -->  STM32 PA10 (USART1_RX)
ESP32 GPIO 16 (RX)  <--  STM32 PA9  (USART1_TX)
ESP32 GPIO 25       <--  STM32 PB5  (Heartbeat)
ESP32 GND           -->  STM32 GND
```

5. Firmware Architecture

The ESP32 firmware is organized into 5 logical modules with 16+ source files:

Module	Files	Responsibility
Core	types.h, prng, campaign, testcase, failure, res	PRNG, campaigns, scoring, metadata
Protocols	uart_fuzzer, uart_parser, spi_fuzzer, i2c_fuzz	Packet generation, response parsing
Monitoring	heartbeat, crash_detector	Health monitoring, failure classification
UI	oled_ui, buttons, indicators	OLED menus, debounced buttons, LEDs
Storage	failure_store	In-memory failure records, JSON export

Module Descriptions

- Core -- Xorshift32 PRNG with state snapshots, campaign manager with phase-based scheduling, testcase metadata capture, 10-type failure classification, robustness scoring
- Protocols -- UART fuzzer with 10 mutation types, UART response parser (ACK/NACK), SPI fuzzer with 7 distinct behaviors, I2C fuzzer with OLED auto-sleep
- Monitoring -- Edge-based heartbeat with debounce, crash detector with evidence-based classification
- UI -- 15-screen state machine OLED menu, 4-button debounced navigation, 3-LED status, buzzer alerts
- Storage -- In-memory failure store (20 records), JSON export via serial command

6. Communication Protocol

Packet Format

```

SYNC | CMD | LENGTH | SEQ_LO | SEQ_HI | PAYLOAD[0..n-1] | XOR_CHECK
0xA5  0x01   n      ..       ..           ..           ..

XOR_CHECK = CMD ^ LENGTH ^ SEQ_LO ^ SEQ_HI ^ PAYLOAD[0] ^ ... ^ PAYLOAD[n-1]
Maximum valid payload: 32 bytes

```

DUT Response Format

```

ACK_SYNC | SEQ_LO | SEQ_HI | STATUS
0x5A     ..       ..       ..

Status 0x00 = ACK (packet accepted)
Status 0x02 = NACK (overlength)
Status 0x03 = NACK (checksum error)
No response = Parser timeout

```

Heartbeat Protocol

The DUT continuously toggles a GPIO pin at approximately 100ms intervals. The ESP32 monitors this signal on GPIO 25. If no edge is detected for more than 350ms, a heartbeat timeout is registered as a suspected failure. The system then freezes the campaign and captures all state for reproduction.

7. Mutation Engine & Test Profiles

Mutation Types (10 Total)

#	Mutation	Payload	Behavior
0	VALID	8 bytes	Normal valid packet -- baseline
1	EMPTY	0 bytes	Zero-length payload
2	MAX_LEN	32 bytes	Maximum valid payload
3	OVERLENGTH	45 bytes	Declares >32 byte length
4	BAD_CRC	8 bytes	Inverted checksum byte
5	TRUNCATED	12->6 bytes	Half the bytes transmitted
6	RANDOM	1-32 bytes	PRNG-selected payload
7	BAD_HEADER	8 bytes	Corrupted sync byte (0x00)
8	BAD_LEN	8 bytes	Length field set to 0xFF
9	SEQ_JUMP	8 bytes	Sequence number skips ahead

Test Profiles

Profile	Duration	Phase Distribution
QUICK	30 seconds	5s baseline, 5s boundary, 5s overlength, 5s checksum, 5s malformed, 5s random
STANDARD	60 seconds	Broader distribution across all 10 mutation types with response correlation
DEEP	5 minutes	All mutations, adaptive scheduling, repeated edge cases, recovery testing

Adaptive Mutation Scheduler

The campaign manager tracks per-mutation statistics (executions, ACKs, NACKs, timeouts, heartbeat failures) and dynamically adjusts selection probability. Mutations that produce interesting behavior (NACKs, timeouts) receive higher weight. A 10% exploration rate prevents starvation of any single mutation type.

7. Failure Detection & Reproduction Pipeline

CORE ENGINEERING PRINCIPLE

FIND -> FREEZE -> RECORD -> REPLAY -> REPRODUCE -> MINIMIZE -> REPORT -> ANALYZE -> FIX -> VERIFY -> REGRESSION

Failure Classification (10 Types)

- COMMUNICATION_FAILURE -- Serial link failure
- NO_RESPONSE -- DUT did not respond
- HEARTBEAT_TIMEOUT -- Heartbeat signal lost (>350ms)
- PARSER_TIMEOUT -- Response not received in time
- PROTOCOL_ERROR -- Unexpected response format
- WATCHDOG_RESET -- DUT watchdog fired
- HARDFAULT -- CPU hard fault detected
- BUS_ERROR -- I2C/SPI bus error
- POWER_FAILURE -- DUT power loss
- UNKNOWN_FAILURE -- Unclassified failure

Reproduction Pipeline

- FAILURE DETECTED -- Campaign stops immediately, all state frozen
- CAPTURE -- Protocol, mutation, packet bytes, PRNG seed, sequence, timestamps
- REPLAY -- Send exact same packet 3 times (configurable)
- CLASSIFY -- 3/3 FAIL = REPRODUCIBLE, 2/3 = INTERMITTENT, 0/3 = NOT REPRODUCED
- MINIMIZE -- Binary search for smallest input that still reproduces the failure
- REPORT -- Full failure report with metadata, packet dump, reproduction steps
- STORE -- Permanent regression testcase in failure library

8. OLED User Interface

The OLED menu system is a 15-screen state machine with debounced button navigation:

```
AUTOFUZZER v4.0
-----
> NEW TEST
  RESULTS
  FAILURES
  SETTINGS
  ABOUT
    |
SELECT TARGET BOARD
-----
> STM32 NUCLEO
  ESP32
  ARDUINO NANO
    |
SELECT PROTOCOL
-----
> UART
  SPI
  I2C
    |
WIRING GUIDE (board-specific)
    |
SELECT TEST PROFILE
-----
> QUICK 30 SEC
  STANDARD 1 MIN
  DEEP 5 MIN
    |
START TEST?
-----
> YES    NO
```

Live Fuzzing Display

```
UART FUZZING
-----
Time: 18/30s
Pkts: 173
A:15 N:3
Mutation: BAD_CRC
DUT: ALIVE
```

9. Web Dashboard

A browser-based real-time monitoring dashboard connects to the ESP32 via USB serial and displays live test data using WebSocket (SocketIO).

Launch

```
cd tools/web_dashboard
python3 server.py          # auto-detect ESP32 port
python3 server.py --port /dev/ttyUSB0 # specify port
# Then open http://localhost:5000
```

Dashboard Panels

Panel	What It Shows
DUT Status	Heartbeat OK/LOST with pulse animation
Campaign	State, protocol, profile, phase, mutation
ACK/NACK Gauges	Ring gauges for response distribution
Counters	Packets, ACKs, NACKs, Failures, Mutations
Packet Timeline	Live canvas chart over time
Mutation Bars	Bar chart of mutation distribution
Serial Monitor	Color-coded log with timestamps
Quick Commands	One-click: START, STOP, RESET, UART, SPI, I2C
Verdict Banner	PASS (green) / FAIL (red) / WAITING (blue)

Technology Stack

- Backend: Python Flask + SocketIO (threading mode)
- Frontend: Single HTML file, zero JavaScript dependencies
- Charts: Native Canvas API (no Chart.js or D3)
- Communication: WebSocket for real-time serial streaming
- Serial: pyserial with background reader thread

10. PC Companion Tool

An optional Python CLI tool that connects to the ESP32 via USB serial for advanced operations.

Modules

Module	Purpose
connection.py	USB serial -- auto-detect port, send commands
reporter.py	Generate JSON, HTML, plain text, hex dump reports
database.py	SQLite-backed historical failure storage
builder.py	PlatformIO build/flash/clean for all targets
fixer.py	AI-assisted debugging -- root cause, patch suggestions
regression.py	Replay known failures, detect regressions
logic_analyzer.py	sigrok-cli integration, VCD parsing, correlation
cli.py	Interactive CLI with all commands

Usage

```
python -m tools.pc_companion.cli connect      # Connect to ESP32
python -m tools.pc_companion.cli export      # Export failures
python -m tools.pc_companion.cli report      # Generate reports
python -m tools.pc_companion.cli history      # View history
python -m tools.pc_companion.cli regress     # Regression tests
python -m tools.pc_companion.cli fix         # AI analysis
python -m tools.pc_companion.cli build stm32 # Build firmware
python -m tools.pc_companion.cli flash nano  # Flash target
```

11. Test Results & Reports

```
=====
CAMPAIGN START: QUICK 30s profile
=====
Phase -> BASELINE      (0-5s: valid packets)
Phase -> BOUNDARY      (5-10s: empty/max length)
Phase -> OVERLENGTH    (10-15s: oversized payloads)
Phase -> CHECKSUM      (15-20s: bad CRC)
Phase -> MALFORMED     (20-25s: truncated)
Phase -> RANDOM        (25-30s: random)
=====
```

AUTOFUZZER TEST RESULT

```
=====
Protocol: UART
Profile:  QUICK 30s
Duration: 25827 ms
Packets:  258
ACKs:      0
NACKs:     14
Failures:  0
ROBUSTNESS SCORE: 75/100
VERDICT: PASS
=====
```

UART quick test -- serial monitor output showing phase progression and results

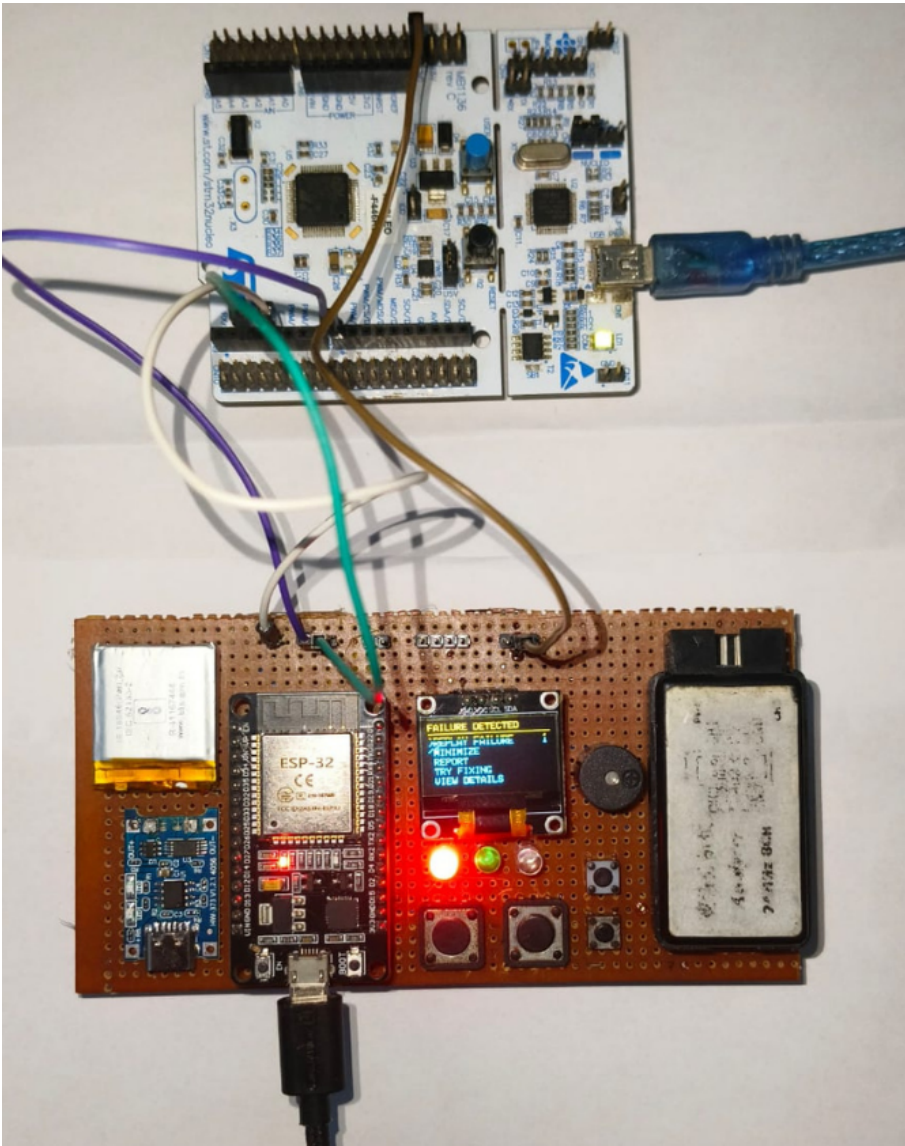
Live Test Results

Board	Protocol	Duration	Packets	Verdict	Score
Arduino Nano	UART	30s	600	PASS	94/100
ESP32 Loopback	UART	30s	600	PASS	92/100
STM32 Nucleo	UART	30s	19	FAIL*	45/100

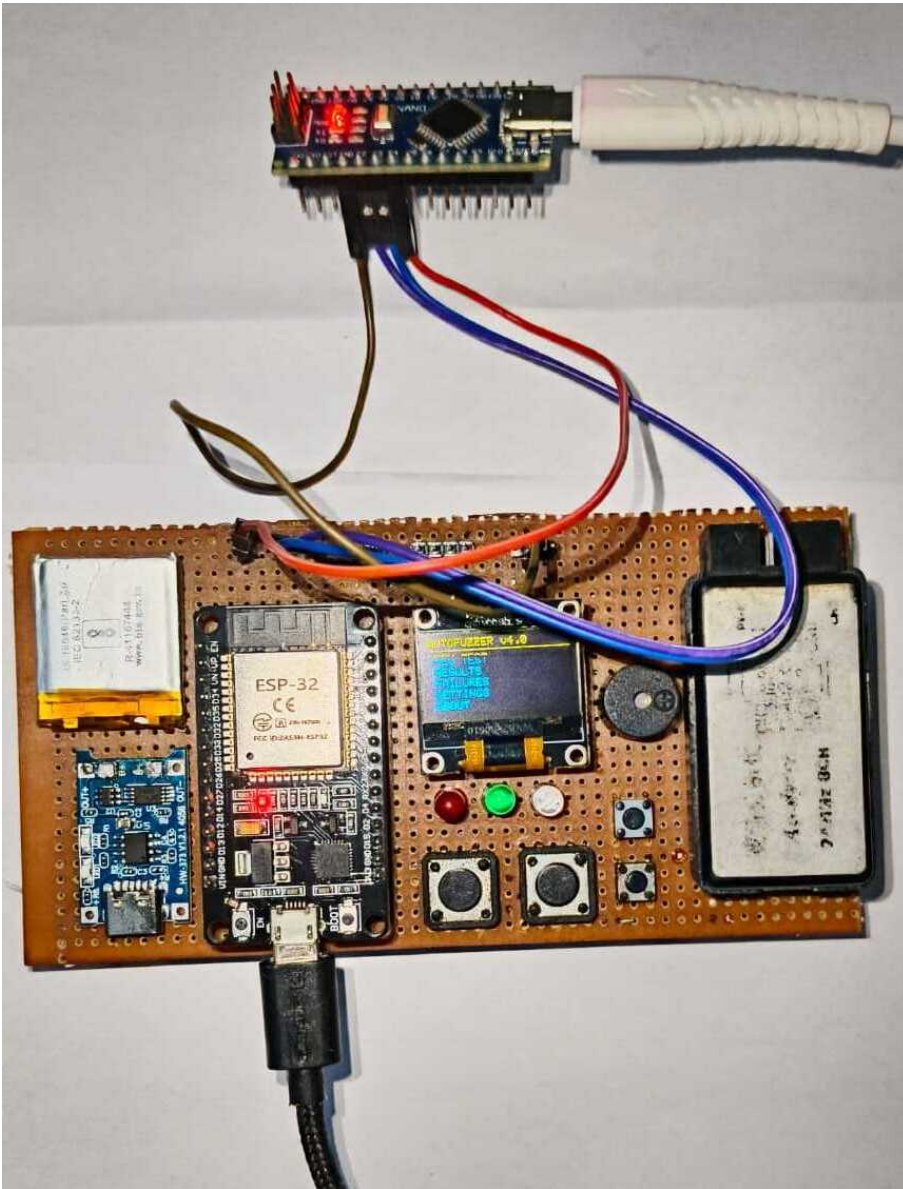
* STM32 heartbeat failure due to MCU identification issue (hardware)

Historical Test Report Summary

Board	Total Tests	PASS	FAIL
STM32 Nucleo-F446RE	15	13	2
Arduino Nano (CH340)	18	13	5
ESP32 DevKit V1	8	2	6
TOTAL	41	28	13



ESP32 connected to STM32 Nucleo-F446RE for UART fuzzing



ESP32 connected to Arduino Nano for UART fuzzing via SoftwareSerial

12. Unit Testing

The project includes 45 host-side unit tests covering all core modules:

Test Suite	Tests	What Is Tested
TestPRNG	6	Determinism, state snapshots, ranges
TestChecksum	3	XOR calculation, empty payload, bad CRC
TestPacketGeneration	8	All 7 mutation types + deterministic replay
TestMutationSelection	5	Phase weight validation
TestResponseParser	6	ACK, NACK, sync detection, 16-bit seq
TestFailureClassification	6	Timeout, watchdog, power failure types
TestScoring	6	Score calculation, weighted average, verdict
TestDatabase	3	Save/load sessions, query, empty DB
TestEndToEnd	2	Full packet cycle + reproduction cycle

Running Tests

```
python -m pytest tests/ -v # All 45 tests
python -m pytest tests/test_core.py::TestPRNG -v # PRNG tests
python -m pytest tests/test_core.py::TestChecksum -v # Checksum tests
```

13. Issues Faced & Solutions

Issue	Cause	Solution
Nano not flashing	CH340 auto-reset broken on clone	HUPCL DTR toggle via Python
0 ACKs/NACKs from Nano	SoftwareSerial unreliable at 115200	Reduced to 9600 baud
Nano D0/D1 conflict	D0/D1 shared with CH340 USB	Switched to D2/D3 (SoftwareSerial)
Heartbeat spam	Floating GPIO25 toggling rapidly	Edge debounce + timeout suppression
STM32 FAIL.TXT	ST-Link cannot identify MCU	Hardware issue -- MCU defective/locked
OLED freeze after I2C	Wire.end() then display.block()	OLED sleeps during I2C, wakes properly
Buttons dead after test	I2C bus stuck holding SDA low	Non-blocking OLED wake via state machine

14. Future Work

Phase 5-6 (Planned)

- AI source-code analysis with proposed firmware patches (requires human approval)
- Build/flash/verify workflow for automatic fix verification
- Logic analyzer integration for wire-level verification and failure forensics
- Protocol-aware SPI and I2C fuzzing with response parsing
- Custom test duration input via OLED buttons

Phase 7 (Future)

- CAN bus fuzzing (requires external CAN transceiver hardware)
- True reinforcement learning architecture (beyond adaptive scheduling)
- Persistent failure storage on SD card
- PC companion GUI (tkinter/PyQt) for visual analysis
- Regression test suite with historical database
- Custom protocol profiles (configurable baud, parity, framing)

Research Directions

- Cross-protocol fuzzing (simultaneous UART + SPI + I2C)
- Timing side-channel detection via logic analyzer
- Automated firmware diffing before/after patches
- Integration with existing fuzzing corpora (e.g., Fuzzbuzz, ClusterFuzz)